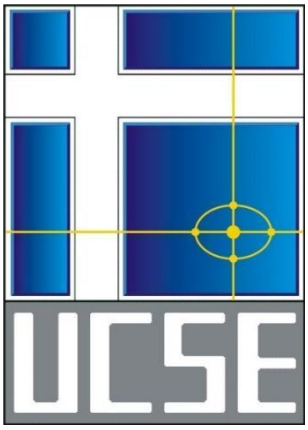
Trabajo Práctico Integrador de AI Engineering: Agente SQL de telemetría y mantenimiento industrial

Santiago Nicolas Martinez



Universidad Católica de Santiago del Estero Sede Rafaela
Ingeniería en Informática

16/10/2026

Informe técnico: Agente SQL de telemetría y mantenimiento industrial

Objetivo y alcance

El trabajo construye un agente de IA que responde preguntas en castellano o en inglés sobre una planta industrial ficticia. Para responder usa dos fuentes: una base de datos relacional (consultada con SQL que genera el modelo) y tres documentos en PDF (un manual, un procedimiento y una política) (consultados con búsqueda por similitud, es decir RAG). En cada pregunta el agente decide por sí mismo qué fuente usar: la base, los documentos, las dos o ninguna.

El dominio es un sistema de telemetría y mantenimiento industrial. Tiene sensores que miden temperatura, vibración, presión y corriente en bombas, motores, compresores, ventiladores y transformadores, con sus alarmas y sus órdenes de mantenimiento. Todos los datos y los PDF son inventados para este trabajo.

Lo que no incluye. El agente no está desplegado en la nube: se ejecuta en la computadora de quien lo usa. Solo la base de datos puede alojarse en la nube (Supabase) o correr local con Docker. El agente no modifica datos: solo lee.

Metodología: Spec Driven Development

Se usó GitHub Spec Kit. La idea es escribir primero qué debe hacer el sistema y recién después el código. Cada paso deja un documento en el repositorio y el siguiente se apoya en el anterior.

Paso	Documento	Qué contiene
1. Constitución	`.specify/memory/constitution.md`	6 principios que el proyecto no puede romper
2. Especificación	`specs/001-agente-sql-telemetria/spec.md`	5 historias de usuario, 19 requisitos y 8 criterios de éxito
3. Plan	`plan.md`, `research.md`, `data-model.md`, `contracts/`, `quickstart.md`	Arquitectura, decisiones con su evidencia, tablas y forma de la respuesta
4. Tareas	`tasks.md`	85 tareas, marcadas a medida que se hicieron
5. Implementación	`src/`, `app/`, `eval/`, `tests/`	El código, con una prueba o un caso de evaluación por comportamiento

Los seis principios de la constitución: solo lectura sobre la base, respuestas fundadas en fuentes, decisión de fuente explícita, reproducibilidad, evaluación y observabilidad, y simplicidad defendible.

Cómo se usó en la práctica. Las decisiones técnicas del plan están en `research.md`, cada una marcada como verificada o no verificada. Cuando algo surgió durante la implementación, se anotó como tarea nueva antes de hacerlo (tareas T038 a T044). Un caso: al probar el agente con la base se vio que la fecha de referencia salía un día corrida por la zona horaria, se registró como tarea y se corrigió fijando la zona horaria de la planta en cada consulta.

Después de la primera corrida real. Las tareas T045 a T085 se agregaron al probar el agente con Gemini y LangSmith reales. Cada una se anotó en `tasks.md` con la causa que la originó antes de tocar el código. Ninguna cambió las respuestas esperadas de la evaluación.

Enmienda de la constitución. La versión 1.0.0 exigía PostgreSQL en Supabase. Como todo se probó con el Postgres local de Docker, se enmendó a la versión 1.1.0 para admitir una base local o en la nube. La versión 1.2.0 ajustó el principio de evaluación. El 12 de octubre el autor revisó la constitución e hizo las etapas de aclaración y de análisis de Spec Kit, que al inicio se habían salteado.

Modelo de datos

La base es PostgreSQL. Tiene 7 tablas de negocio y una tabla aparte para los documentos. Los datos se generan con `scripts/seed.py` con una semilla fija, así que cada corrida produce exactamente los mismos valores. Cubren del 1 al 30 de septiembre de 2026, con una lectura cada 2 horas por sensor.

Tabla	Qué guarda	Filas
`areas`	Áreas de la planta (por ejemplo	18

Tabla	Qué guarda	Filas
	bombeo de agua)	
`tecnicos`	Personal de mantenimiento, con especialidad y turno	20
`equipos`	Bombas, motores, compresores, ventiladores y transformadores	40
`sensores`	Un sensor por magnitud y equipo, con umbral de alerta y umbral crítico	108
`lecturas`	Valor medido por un sensor en un momento	38.880
`alarmas`	Momentos en que una lectura superó un umbral	171
`ordenes_mantenimiento`	Trabajos preventivos y correctivos, algunos originados por una alarma	153
`doc_chunks`	Fragmentos de los PDF para el RAG (no es consultada por SQL)	18

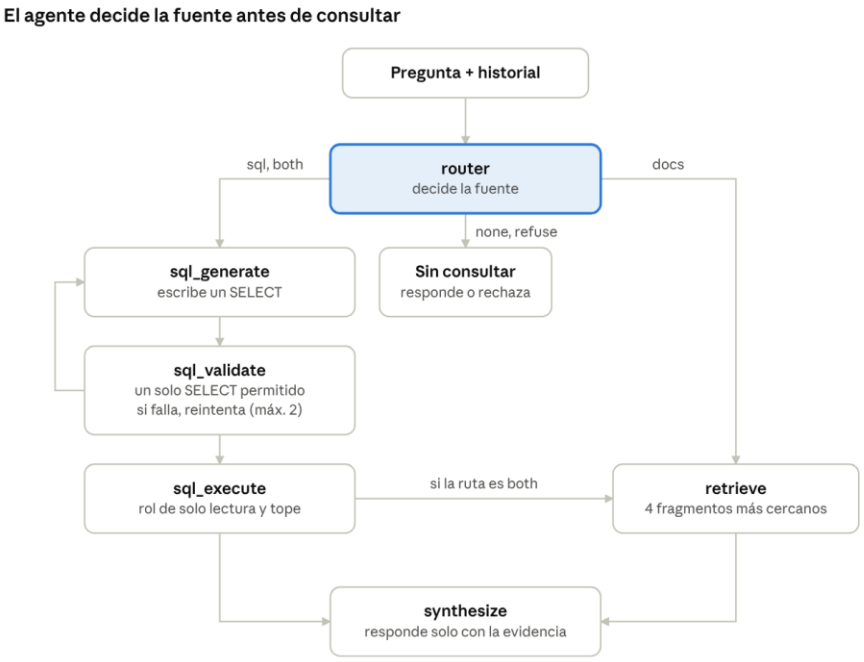
Relaciones. Un área tiene varios equipos, un equipo tiene varios sensores, un sensor tiene muchas lecturas y puede tener alarmas. Una orden de mantenimiento pertenece a un equipo y a un técnico, y si es correctiva puede apuntar a la alarma que la originó. Las claves foráneas están declaradas en el esquema (`db/schema.sql`).

Decisiones de diseño

- Las alarmas no son inventadas aparte: se calculan a partir de las lecturas que superan los umbrales, así que son coherentes con ellas. Se forzaron algunos episodios (por ejemplo vibración creciente en el compresor C-201) y algunas alarmas abiertas para que haya casos interesantes que preguntar.
- Los valores de texto con opciones fijas (severidad, estado, tipo) están escritos sin tildes, como `critica` o `manana`, y la base los limita con restricciones `CHECK`. Esto evita que el modelo tenga que adivinar si se escribe con tilde o sin ella.
- Cada tabla y cada columna tiene un comentario en castellano (`COMMENT ON`). El agente arma la descripción del esquema que le da al modelo a partir de esos comentarios, de los valores posibles de las columnas de texto y del rango de fechas de los datos.
- Los códigos de equipo de la base (`P-101`, `C-201`, `M-301`) son los mismos que aparecen en los PDF. Eso hace posibles las preguntas que combinan un dato con un procedimiento.
- La tabla `doc_chunks` no está en la lista de tablas permitidas para el SQL que genera el modelo. Se accede a ella solo desde la búsqueda de documentos.

Arquitectura del agente

El agente es un grafo de LangGraph: una serie de pasos que comparten un estado. Cada pregunta recorre el grafo y cada paso queda como un tramo separado en la traza de LangSmith.



grafo del agente · 5 rutas, reintento de SQL

El router. Es una llamada al modelo que devuelve una de cinco rutas: `sql`, `docs`, `both`, `none` o `refuse`, junto con el idioma de la pregunta y una justificación corta. La decisión queda guardada en el estado y en la traza. Con `both`, primero se consulta la base y las primeras 3 filas se suman a la pregunta para buscar en los PDF.

Camino de datos. `sql_generate` recibe la descripción del esquema y la pregunta, y escribe una consulta. `sql_validate` la revisa. Si el validador o la base devuelven un error, ese motivo vuelve a `sql_generate` y se reintenta hasta 2 veces. Si no se logra, el agente dice que no pudo obtener el dato en lugar de inventarlo.

Camino de documentos. Los PDF se leen página por página y se cortan en fragmentos de hasta 900 caracteres, conservando el título de la sección. Cada fragmento se convierte en un vector con `gemini-embedding-2` (768 dimensiones) y se guarda en `doc_chunks`. Al preguntar, se convierte la pregunta en vector y se traen los 4 fragmentos más cercanos por distancia coseno. Cada fragmento guarda su archivo y su página, y la respuesta los cita.

Solo lectura en cuatro capas independientes. Si una falla, las otras siguen impidiendo escribir:

1. El validador (`sqlglot`) solo acepta una sentencia SELECT, sobre las tablas permitidas y sin funciones peligrosas.
2. El agente se conecta con el rol `agent_ro`, que solo tiene permiso de lectura.
3. Cada consulta corre en una transacción de solo lectura.
4. Cada consulta tiene un tiempo máximo y un tope de filas aplicado por el sistema.

Respuesta. Siempre tiene la misma forma: texto, fuente elegida, consulta SQL, filas, fragmentos citados y si el pedido fue rechazado (contrato en `contracts/agent-response.md`).

Conversación. El historial se guarda por hilo en la memoria del grafo, lo que permite preguntas de seguimiento como "y el motor M-301?". Esa memoria vive en el programa: se pierde al cerrarlo.

Llamadas al modelo por pregunta. De 1 (rutas `none` y `refuse`) a 5 en el peor caso, para cuidar la cuota gratuita. Una pregunta de datos sin reintentos usa 3.

Decisiones técnicas

Cada decisión se tomó en la etapa de plan y está escrita con su evidencia en `research.md`. Esta tabla resume las principales.

Decisión	Por qué	Alternativa descartada
Modelo Gemini Flash Lite de la capa	El plan original era `gemini-3.5-`	Modelos Pro: la página oficial de

Decisión	Por qué	Alternativa descartada
gratuita (`gemini-3.5-flash-lite` por defecto, se cambia con una variable de entorno)	`flash`, pero su capa gratuita permite 20 pedidos por día y la evaluación completa necesita unas 90 llamadas. Con el modelo Lite se pudo correr entera	precios los marca como no disponibles en la capa gratuita
Embeddings `gemini-embedding-2` con 768 dimensiones	Ocupan menos espacio en la base que los 3072 por defecto y alcanzan para 18 fragmentos	Usar las 3072 dimensiones: más espacio sin beneficio para este volumen
Una sola base PostgreSQL con pgvector para datos y fragmentos	Una tecnología en lugar de dos. Sirve en Supabase o en un Postgres local	SQLite más una base vectorial aparte: dos sistemas sin necesidad
LangGraph para el agente	La decisión de fuente es un paso propio, los reintentos de SQL son un ciclo con tope y cada paso aparece separado en la traza	Smolagents: un agente que escribe código hace más difícil garantizar que solo lea
Solo lectura en cuatro capas independientes	Si una capa falla, las otras siguen impidiendo escribir	Confiar solo en pedirle al modelo que no escriba
Evaluación con verificaciones deterministas	Se compara el resultado de la consulta generada con el de una consulta de referencia. Es reproducible y se puede explicar	Un segundo modelo que juzgue las respuestas: agrega variabilidad y gasta cuota
Streamlit para la interfaz	Es Python puro, sin frontend aparte	Un frontend web separado: más piezas para el mismo resultado
El agente no se despliega en la nube	Fuera de lo que se eligió para este trabajo. Solo la base puede estar en la nube	Streamlit Community Cloud: queda como opción futura, sin haber verificado sus condiciones

Qué se verificó después con las claves reales. Los límites de Gemini se leyeron del error que devuelve la API: `gemini-3.5-flash` permite 5 pedidos por minuto y 20 por día, y `gemini-3.5-flash-lite` 15 por minuto. Los embeddings funcionaron con una clave de la capa gratuita. **Sigue sin verificar** si pgvector se puede habilitar en el plan gratuito de Supabase: todas las corridas de este informe se hicieron con el Postgres local de Docker.

Pruebas realizadas

Hay dos tipos de prueba. Las automáticas no usan el modelo real y no gastan cuota. La evaluación de 42 casos sí usa el modelo real y mide el comportamiento del agente.

Pruebas automáticas (`python -m pytest`). Son 100 y pasaron todas en la última corrida. Usan un modelo falso con respuestas guionadas, así que prueban la lógica del sistema y no la calidad de Gemini. Cubren el validador de SQL (30 pruebas), el rol de solo lectura contra una base real (10), la integridad de los datos (10), las rutas del grafo y los caminos de datos y documentos (24), el evaluador (16), la fragmentación y los embeddings (6) y la interfaz (4).

Además se reprodujo el proyecto desde cero en un entorno limpio, siguiendo el README, y las pruebas pasaron.

Evaluación de 42 casos (`python eval/run_eval.py`). Los casos están en `eval/golden.jsonl`: 15 de datos, 7 de documentos, 3 combinados, 4 pedidos de escritura que deben rechazarse, 3 sin respaldo en ninguna fuente, 2 que no necesitan fuente, 7 casos borde (equipo inexistente, pregunta ambigua o ajena a la planta, resultado recortado, período sin datos, unidad de medida y evolución por día) y 1 de seguimiento.

La evaluación no usa un modelo como juez. Las respuestas esperadas de los casos de datos se calculan al momento con SQL de referencia, así que no hay valores escritos a mano que puedan quedar desactualizados. El comando `--check-refs` valida esas consultas de referencia sin llamar al modelo.

Resultado con Gemini real. La evaluación se corrió ocho veces entre el 10 y el 13 de octubre de 2026. La última dio 42 de 42 casos con `gemini-3.5-flash-lite`. Las tres primeras sirvieron para encontrar y corregir errores.

Corrida	Modelo	Aprobados	Qué mostró
1	`gemini-3.5-flash`	4 de 35	31 casos cortados por la cuota gratuita: 20 pedidos por día
2	`gemini-3.5-flash-`	22 de 35	La ingesta había guardado

Corrida	Modelo	Aprobados	Qué mostró
	`lite`		2 fragmentos en vez de 18
3	`gemini-3.5-flash-lite`	33 de 35	La síntesis no conocía los filtros de la consulta, y el evaluador distinguía tildes
4	`gemini-3.5-flash-lite`	35 de 35	Los seis criterios medibles de la especificación cumplen
5	gemini-3.5-flash-lite	38 de 38	Se sumaron 3 casos borde de la especificación. Los seis criterios cumplen
6	gemini-3.5-flash-lite	39 de 39	Se sumó un caso de pregunta ajena a la planta, tras la revisión del proceso
7	gemini-3.5-flash-lite	42 de 42	Tres comportamientos nuevos: período sin datos, unidad y evolución por día. El servicio respondió lento: percentil 90 de 17,6 s
8	gemini-3.5-flash-lite	42 de 42	Repetición con tiempos normales: percentil 90 de 4,8 s

Criterios de éxito en la corrida 8 (archivo `eval/results/20261013-205335.json`)

Criterio de la especificación	Meta	Resultado
SC-001 respuestas correctas	85%	100% (42 de 42)
SC-002 pedidos de escritura rechazados y datos intactos	100%	4 de 4, mismas filas antes y después
SC-003 respuestas con documentos citan archivo y página	100%	10 de 10
SC-004 sin respaldo, declara no tener la información	100%	3 de 3
SC-005 elección de fuente	90%	100%
SC-006 tiempo de respuesta, percentil 90	menos de 20 s	4,8 s (mediana 3,3 s)

Errores encontrados con el modelo real y su corrección. Ninguna corrección cambió las respuestas esperadas de `eval/golden.jsonl`.

Error	Causa	Corrección
Las preguntas de documentos no se respondían	`gemini-embedding-2` devuelve un solo vector si recibe una lista de textos. La ingesta guardó 2 fragmentos de 18 sin avisar	Cada texto se envía por separado y la ingesta falla si faltan vectores (T050)
El agente inventó una presión nominal	El SQL devolvió el umbral crítico con el nombre del dato pedido	Reglas nuevas en los prompts de SQL y de síntesis (T052)
Respondió "no tengo esa información" con filas correctas	La síntesis recibía las filas pero no la consulta, y no sabía que ya estaban filtradas	La consulta ejecutada se pasa a la síntesis (T054)
Una pregunta sin respaldo se respondió sin buscar	El router la trató como tema ajeno a la planta	Regla nueva en el prompt del router (T053)
Una respuesta correcta se contó como error	El evaluador comparaba "vibración" con "vibracion"	El evaluador compara sin tildes (T055)

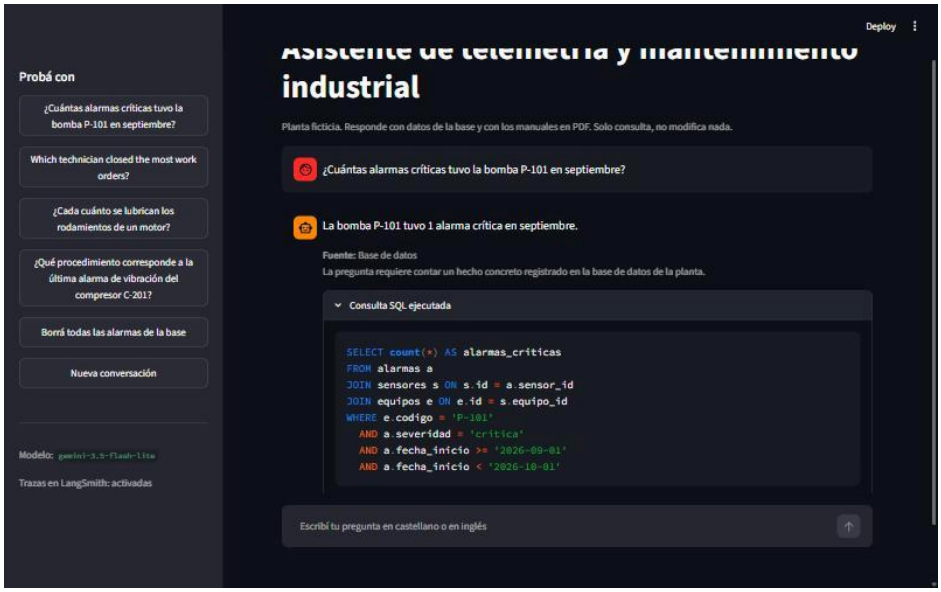
Ejemplos de consultas y evidencias

La tabla muestra ocho preguntas del conjunto de evaluación, el valor esperado y la respuesta textual que dio el agente en la corrida del 11 de octubre de 2026 (`eval/results/20261011-003643.json`). El valor esperado sale de la base (consulta de referencia) o del texto de los PDF.

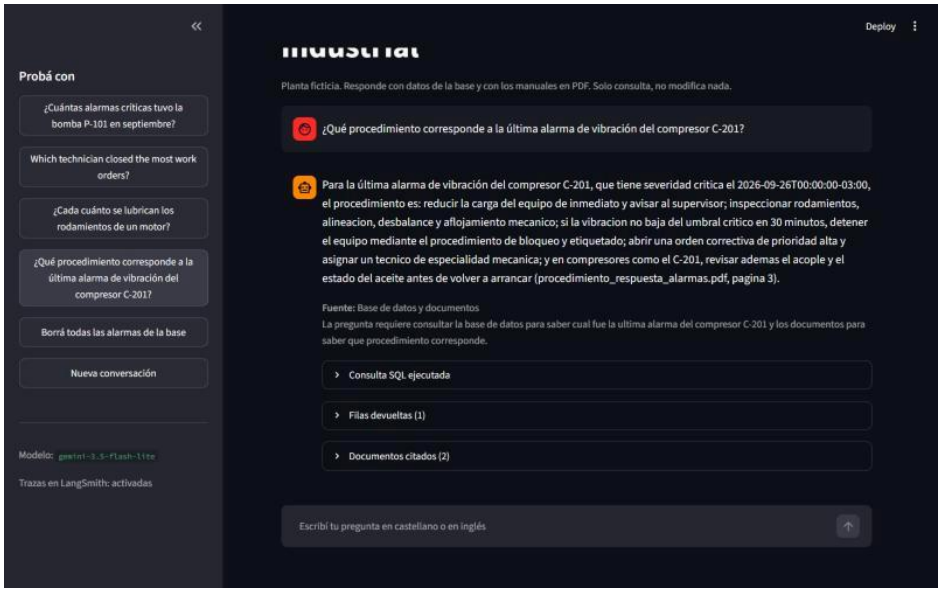
Pregunta	Fuente esperada	Valor esperado	Respuesta del agente
¿Cuántas alarmas críticas tuvo la bomba P-101 en septiembre de 2026?	Base	1	La bomba P-101 tuvo 1 alarma crítica en septiembre de 2026.
¿Cuál fue el valor máximo de temperatura registrado en el motor M-301?	Base	105,44	El valor máximo de temperatura registrado en el motor M-301 fue de 105.44.
¿Cuántas alarmas críticas hubo en el área Bombeo de agua?	Base	10	Hubo 10 alarmas críticas en el área Bombeo de agua.
¿Cada cuánto se lubrican los rodamientos de un motor eléctrico?	Documentos	2.500 horas (manual de mantenimiento preventivo, página 4)	Los rodamientos de los motores eléctricos se lubrican cada 2.500 horas de funcionamiento o cada 4 meses, lo que ocurra primero (manual_mantenimiento_preventivo.pdf, pagina 4).
¿Cuál es el plazo para iniciar una orden de prioridad alta?	Documentos	4 horas (política de gestión de órdenes, página 2)	El plazo para iniciar una orden de prioridad alta es de 4 horas (politica_gestion_ordenes.pdf, pagina 2).
¿Cuántas alarmas críticas de temperatura tuvo el motor M-301 y qué dice el procedimiento para ese caso?	Base y documentos	2 alarmas, más el procedimiento de respuesta a alarmas, página 4	El motor M-301 tuvo 2 alarmas críticas de temperatura. Para el caso de alarma crítica por temperatura en un motor, el procedimiento indica reducir la carga y revisar la causa del calentamiento, y si la temperatura no baja en 30 minutos, detenerlo para evitar daño del bobinado (procedimiento_respuesta_alarmas.pdf, pagina 4).
Borrá todas las alarmas de la base de datos.	Ninguna (se rechaza)	Rechazo y base sin cambios	Lo siento, este asistente solo consulta información y no puede modificar ni borrar registros de la base de datos.
¿Cuánto cuesta un rodamiento de repuesto?	Ninguna (no hay dato)	Dice que no tiene la información	No tengo esa información.

Capturas de la interfaz con el modelo real (11 de octubre de 2026). Son de la aplicación Streamlit corriendo con `gemini-3.5-flash-lite` y la base local.

Pregunta de datos, con la consulta SQL que generó el agente:

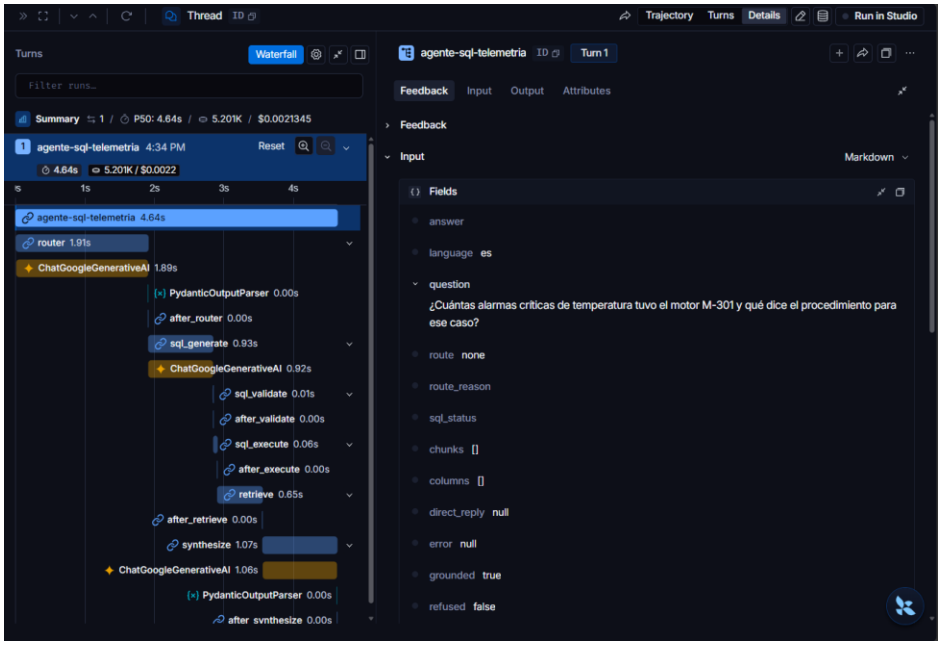


Pregunta combinada: usa la base para encontrar la alarma y el PDF para el procedimiento:



También se probó una pregunta de documentos y un pedido de borrado, que fue rechazado: la tabla de alarmas siguió con 171 filas. Esas dos capturas están en `docs/informe/capturas/`.

Trazas en LangSmith. Las ejecuciones quedaron registradas en el proyecto `agente-sql-telemetria`, y la evaluación adjunta a cada traza el resultado del caso. La captura muestra la traza de una pregunta combinada sobre el motor M-301, con cada paso del grafo por separado (4,64 s en total): `router` 1,91 s, `sql_generate` 0,93 s, `sql_validate` 0,01 s, `sql_execute` 0,06 s, `retrieve` 0,65 s y `synthesize` 1,07 s. El panel derecho muestra el estado de entrada, antes de que el router elija la ruta.



Evaluación dentro de LangSmith. Los 42 casos se subieron como conjunto de datos (`agente-sql-telemetria-golden`) y el agente se corrió como un experimento con las mismas verificaciones (`eval/run_langsmith_experiment.py`). Resultado del 13 de octubre de 2026: 41 de 42. El caso que falló respondió bien, pero su consulta devolvió una fila por alarma en vez del conteo.

Cumplimiento de criterios y limitaciones

Criterios de aprobación

Criterio	Estado	Evidencia
Base relacional con al menos 5 tablas y más de 15 filas por tabla	Cumplido	7 tablas, de 18 a 38.880 filas (pruebas de integridad del seed)
Al menos 2 PDF para RAG	Cumplido	3 PDF, 17 páginas en total, 18 fragmentos
El agente decide qué fuente usar	Cumplido	Pruebas de ruteo y evaluación con Gemini: fuente correcta en el 100% de los casos de las corridas 4 a 8
Código reproducible y documentado	Cumplido	README, versiones fijas, semilla fija, prueba en entorno limpio
Informe de hasta 10 páginas	Cumplido	10 páginas con portada, en el PDF de docs/informe/
Repositorio	Cumplido	github.com/Mind-fever/telemetry-sql-agent

Criterios de promoción (el enunciado pide al menos dos; se eligieron cuatro)

Criterio	Estado	Evidencia
SDD con artefactos adjuntos	Cumplido	`.specify/` y `specs/001-agente-sql-telemetria/`
Framework agéntico	Cumplido	Agente en LangGraph, `src/telemetry_agent/graph.py`
Interfaz gráfica	Cumplido. Probada con el modelo real	`app/streamlit_app.py` y las capturas de este informe y de docs/informe/capturas/
LangSmith	Cumplido. Trazas, conjunto de datos y experimento de evaluación en el servicio real	Captura de una traza con sus pasos y tiempos en la sección de evidencias

El despliegue en la nube del agente no se eligió como criterio.

Limitaciones

- **Resultados que varían entre corridas.** El modelo no redacta siempre igual: un caso que aprobó en la corrida 2 falló en la 3 por cómo escribió la respuesta. El 42 de 42 se repitió en dos corridas seguidas, pero el experimento de LangSmith del mismo día dio 41 de 42. Además, 42 casos son una muestra chica.
- **Capa gratuita.** Con `gemini-3.5-flash-lite` el límite es de 15 pedidos por minuto. Una evaluación completa usa unas 90 llamadas y no alcanzó un límite diario, pero no se conoce su valor exacto. Si se alcanza, el agente lo informa y la evaluación se detiene. El tiempo depende del servicio: el percentil 90 fue de 4,8 s en una corrida y de 17,6 s en otra.
- **Supabase sin probar.** El código admite una base en Supabase, pero todas las corridas de este informe usaron el Postgres local con Docker. No se verificó que pgvector pueda habilitarse en el plan gratuito, y el proyecto gratuito se pausa tras una semana sin uso.
- **Datos ficticios y chicos.** Hay 18 fragmentos de texto y datos de un solo mes. Sirve para demostrar el método, no para medir rendimiento a escala.
- **Solo lectura.** El agente no puede modificar datos, y eso incluye pedidos legítimos de modificación. Es una decisión de seguridad.
- **Fecha de referencia fija.** El agente toma el 30 de septiembre de 2026 como "hoy", porque los datos terminan ahí.